F1 Analytics & Prediction Platform — Complete Walkthrough

Project Architecture

```
F1 prediction/
├── data/                    # Raw CSV files (8 datasets)
├── src/                     # Core Python modules
│   ├── __init__.py
│   ├── data_loader.py       # Loads & merges all CSV data
│   ├── feature_engineering.py  # Creates ML features
│   ├── models.py            # Model training utilities
│   └── visualization.py     # Chart-making helpers
├── notebooks/               # 9 analysis scripts
│   ├── 01_data_exploration.py
│   ├── 02_race_winner_prediction.py
│   ├── 03_driver_performance_analysis.py
│   ├── 04_strategy_optimization.py
│   ├── 05_championship_prediction.py
│   ├── 06_goat_ranking.py
│   ├── 07_driver_clustering.py
│   ├── 08_race_simulation.py
│   └── 09_constructor_prediction.py
├── dashboard/               # Streamlit interactive web app
│   ├── app.py               # Main dashboard entry point
│   └── pages/               # 6 sub-pages
├── models/                  # Saved trained ML models (.pkl)
├── outputs/                 # Generated charts and CSVs
└── requirements.txt         # Python dependencies
```

How Each Part Works (Beginner-Friendly)

Core Module: `data_loader.py`

What it does: Loads 8 CSV files and merges them into one big table (DataFrame).

Key concept — DataFrame: Think of it like a giant Excel spreadsheet in Python. Each row is one driver's result in one race; each column is a piece of information (driver name, position, points, etc.).

Key concept — pd.merge(): This is like VLOOKUP in Excel. It connects two tables using a shared column. For example, we connect race results to driver information using `driver_id`.

The merging pipeline:

```
results (who finished where)
   + races (which race, what date)
```

```
    + drivers (driver name, nationality)
    + constructors (team name)
    + circuits (track name, location)
    + qualifying (qualifying positions)
    = MASTER DataFrame (25,939 rows × 40+ columns)
```

Core Module: `feature_engineering.py`

What it does: Creates "features" — the numbers that ML models use to make predictions.

Key concept — Feature: A feature is any measurable property. For example, "average finish position in last 5 races" is a feature. Models learn patterns from features.

Key concept — Data Leakage: We must NOT give the model information about the future. We use `shift(1)` to ensure features only use *past* data.

Features created:

Feature	What It Means
`driver_avg_pos_last3`	Average finishing position in the last 3 races
`driver_avg_pos_last5`	Average finishing position in the last 5 races
`cumulative_win_rate`	Career win percentage up to that point
`constructor_avg_points_last5`	How well the team has scored recently
`circuit_experience`	How many times a driver has raced at this track
`season_momentum`	Rolling points trend this season

Notebook-by-Notebook Breakdown

Notebook 01: Data Exploration

Purpose: Understand the shape and patterns in the data before building models.

What it does:

- Counts total races per season (chart: `01_races_per_season.png`)
- Shows top 20 winners of all time (chart: `01_top20_wins.png`)
- Plots grid position vs. finish position (chart: `01_grid_vs_finish.png`)
- Analyzes DNF rates by decade (chart: `01_dnf_by_decade.png`)
- Examines points distribution (chart: `01_points_distribution.png`)

Key takeaway: Grid position strongly correlates with finishing position — starting at the front matters a lot!

Notebook 02: Race Winner Prediction (Supervised Classification)

Purpose: Predict whether a driver will win a race (yes/no).

Algorithm: XGBoost (eXtreme Gradient Boosting)

Think of it like this:

1. Imagine you ask 100 different "experts" (decision trees) to vote on whether a driver will win
2. Each expert looks at different combinations of features
3. The final answer is the majority vote

How it works step by step:

1. **Load data** → 25,939 race results
2. **Create features** → Rolling averages, win rates, circuit experience
3. **Split data** → Train on races before 2018, test on 2018+ (time-based split)
4. **Train 5 models** → Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost
5. **Evaluate** → Compare using ROC-AUC score (higher = better)
6. **Save best model** → `models/xgboost.pkl`

Models compared:

Model	Type	How It Works (Simple)
Logistic Regression	Linear	Draws a straight line to separate winners from non-winners
Random Forest	Ensemble	100 decision trees vote together
XGBoost	Boosting	Trees built sequentially, each fixing the previous one's mistakes
LightGBM	Boosting	Like XGBoost but faster with large data
CatBoost	Boosting	Like XGBoost but handles categorical data better

Output: Feature importance chart (`02_feature_importance.png`), Model comparison (`02_model_comparison_auc.png`)

> **Want to swap the algorithm?** In `notebooks/02_race_winner_prediction.py`, look for the `models` dictionary (around line 90). You can add any scikit-learn compatible classifier. For example, to try a Support Vector Machine:
>
> ```
> from sklearn.svm import SVC
> models['svm'] = SVC(probability=True)
> ```

Study Resources:

- XGBoost Official Docs
- StatQuest: XGBoost (YouTube)
- scikit-learn Classifiers

Notebook 03: Driver Performance Analysis

Purpose: Create career profiles and compare teammates.

What it does:

- Calculates consistency index (standard deviation of finishing positions)
- Tracks overtaking ability (position changes per race)
- Builds head-to-head teammate comparison

Key concept — Standard Deviation: Measures how much a driver's results vary. Low std = consistent (always finishes in similar positions). High std = inconsistent (sometimes great, sometimes bad).

Output: Teammate head-to-head (`03_teammate_h2h.png`), Top overtakers (`03_top_overtakers.png`)

Notebook 04: Strategy Optimization

Purpose: Analyze how different race strategies (tyre choices, pit stops) affect outcomes.

Algorithm: Monte Carlo Simulation

Think of it like rolling dice thousands of times:

1. Simulate a race 1000 times
2. In each simulation, randomize things like safety cars, weather, pit stop timing
3. Count how often each strategy wins
4. The strategy that wins most often is probably the best

Key concept — Monte Carlo: Named after the casino in Monaco. Instead of calculating exact probabilities (which is very hard), we simulate thousands of random scenarios and count the results.

Output: Strategy simulation results (`04_simulation_results.png`), Grid-to-points expectation (`04_grid_points_expectation.png`)

> **Want to change the simulation?** In `notebooks/04_strategy_optimization.py`, find the `simulate_race()` function. You can modify:
>
> - `n_simulations`: More simulations = more accurate but slower
> - Safety car probability
> - Tyre degradation rates

Study Resources:

- Monte Carlo Simulation Explained (YouTube)
- Wikipedia: Monte Carlo Method

Notebook 05: Championship Prediction

Purpose: Given mid-season standings, predict who will win the championship.

Algorithm: Random Forest Classifier

Like XGBoost's cousin — it also uses many decision trees, but each tree is built independently (not sequentially).

Features used:

Feature	Meaning
`standing_pos`	Current championship position
`cum_points`	Total points accumulated so far
`gap_to_leader`	Points behind the championship leader
`races_remaining`	How many races are left
`avg_points_per_race`	Average points scored per race

How backtesting works: We "pretend" we're at Round 11 of 2021, give the model the standings at that point, and ask "who will win?" Then we check against reality.

Output: Championship progression chart (`05_championship_progression_2021.png`)

> **Want to swap the algorithm?** In `notebooks/05_championship_prediction.py`, line ~127:
>
> ```
> # Current: Random Forest
> clf = RandomForestClassifier(n_estimators=100, max_depth=5)
>
> # Alternative: Gradient Boosting
> from sklearn.ensemble import GradientBoostingClassifier
> clf = GradientBoostingClassifier(n_estimators=100, max_depth=3)
> ```

Study Resources:

- StatQuest: Random Forests (YouTube)
- scikit-learn Random Forest

Notebook 06: GOAT Ranking

Purpose: Create a customizable "Greatest of All Time" ranking system.

Algorithm: Weighted Composite Score (not ML, but data-driven)

This is simpler than ML — it's a formula:

```
GOAT_Score = (w1 × win_rate) + (w2 × championships) + (w3 × podium_rate)
           + (w4 × avg_finish) + (w5 × reliability)
```

Each metric is normalized to 0-1 scale, then multiplied by user-chosen weights.

Key concept — Min-Max Normalization:

```
normalized = (value - min) / (max - min)
```

This converts any number to a 0-1 scale so different metrics can be compared fairly.

Output: GOAT ranking chart (`06_goat_ranking.png`)

> **Want to add new criteria?** In `notebooks/06_goat_ranking.py`, add new metrics to the `features` list and their weights.

Notebook 07: Driver Clustering (Unsupervised Learning)

Purpose: Group drivers into "archetypes" without pre-defined labels.

Algorithm: K-Means Clustering

Imagine throwing 258 dots on a 2D map. K-Means finds 4 natural groups:

1. Pick 4 random "center points"
2. Assign each dot to its nearest center
3. Move each center to the middle of its assigned dots
4. Repeat steps 2-3 until centers stop moving

Archetypes discovered:

Archetype	Description
Dominators & Champions	High win rate, low average position
Consistent Scorers	Mid-range positions, reliable finishers
High-DNF Era	Older era drivers with high retirement rates
Backmarkers	Rarely scored points, high average position

Algorithm: PCA (Principal Component Analysis)

Used for visualization. Our drivers have 6 feature dimensions (win_rate, podium_rate, etc.). PCA squashes these 6 dimensions into 2D so we can plot them on a scatter chart.

Output: Cluster scatter plot (`07_driver_clustering.png`), Cluster data (`driver_clusters.csv`)

> **Want to try a different clustering algorithm?** In `notebooks/07_driver_clustering.py`, swap K-Means for DBSCAN:
>
> ```
> from sklearn.cluster import DBSCAN
> clustering = DBSCAN(eps=1.5, min_samples=5)
> labels = clustering.fit_predict(X_scaled)
> ```
>
> DBSCAN automatically determines the number of clusters and can find oddly-shaped groups.

Study Resources:

- StatQuest: K-Means Clustering (YouTube)
- StatQuest: PCA (YouTube)
- scikit-learn Clustering

Notebook 08: Race Simulation

Purpose: Simulate full races with lap-by-lap overtaking and DNFs.

Algorithm: Monte Carlo Simulation (lap-level)

Each simulation:

1. Start drivers in grid order
2. For each lap:
    - Check if any driver retires (random chance based on historical DNF rate)
    - Check if faster drivers overtake slower ones (based on pace difference × track difficulty)
3. Record the winner
4. Repeat 500-2000 times
5. Win probability = times_won / total_simulations

Key parameter — overtaking_factor: Controls how easy it is to pass:

- Low (0.1) = Monaco-style circuit (very hard to pass)
- High (1.0) = Spa-style circuit (easy to pass)

Output: Race comparison chart (`08_race_comparison.png`)

Notebook 09: Constructor Prediction

Purpose: Predict which constructor (team) will dominate in a season.

Algorithm: Gradient Boosting Regressor

Similar to XGBoost but predicts a continuous number (points) instead of a category (win/lose).

Features: Team's historical performance, budget proxy, driver lineup strength, circuit-specific performance.

Study Resources:

- Gradient Boosting Explained (YouTube)
- scikit-learn Gradient Boosting

Interactive Dashboard

The Streamlit dashboard provides an interactive frontend for all models. It is currently running at:

> **http://localhost:8501**

Dashboard Pages:

Page	What It Does
Home	Overview metrics and navigation
Race Prediction	Select drivers, predict win probabilities using trained XGBoost
Driver Analysis	Browse any driver's career stats and finish distribution
Race Simulation	Run Monte Carlo simulations with adjustable parameters
Championship	Mid-season championship forecasting
GOAT Index	Custom-weighted all-time ranking with adjustable sliders
Driver Clustering	Interactive scatter plot of driver archetypes

How to run the dashboard:

```
streamlit run dashboard/app.py
```

How to Change Any Algorithm — Quick Reference

Notebook	Current Algorithm	Where to Change	Alternative Ideas
02	XGBoost	Line ~90 `models` dict	SVM, Neural Network, Naive Bayes
05	Random Forest	Line ~127 `clf = ...`	Gradient Boosting, Logistic Regression
07	K-Means	Line ~87 `kmeans = ...`	DBSCAN, Agglomerative, Spectral
08	Monte Carlo Sim	`simulate_race()` function	Agent-based modeling
09	Gradient Boosting	Model definition section	XGBoost, LightGBM, Linear Regression

General Steps to Swap an Algorithm:

1. **Import** the new algorithm from scikit-learn
2. **Replace** the model creation line (e.g., `clf = NewModel(...)`)
3. **Re-run** the notebook to train and evaluate
4. The rest of the pipeline (features, evaluation, saving) stays the same!

Recommended Study Resources for Beginners

Free Courses:

- Google's Machine Learning Crash Course
- Kaggle Learn (Free)
- StatQuest YouTube Channel — Excellent visual explanations

Books:

- "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurelien Geron
- "Python Data Science Handbook" by Jake VanderPlas (free online)

Key Libraries Documentation:

- pandas — Data manipulation
- scikit-learn — Machine learning
- matplotlib — Charts and visualization
- Streamlit — Interactive dashboards

Verification Summary

Component	Status	Notes
Data loading	Verified	25,939 rows, 8 CSV files merged
Feature engineering	Verified	17 features with no data leakage
Notebook 01 (Exploration)	Verified	5 charts generated
Notebook 02 (Race Prediction)	Verified	XGBoost best performer, 5 models saved
Notebook 03 (Driver Analysis)	Verified	Performance profiles generated
Notebook 04 (Strategy)	Verified	Monte Carlo simulation working
Notebook 05 (Championship)	Verified	Model saved, backtested on 2021
Notebook 06 (GOAT Ranking)	Verified	Rankings generated
Notebook 07 (Clustering)	Verified	4 archetypes found, CSV saved
Notebook 08 (Race Simulation)	Verified	Lap-level simulation working
Notebook 09 (Constructor)	Verified	Constructor prediction working
Dashboard	Verified	Running on localhost:8501, all 6 pages functional